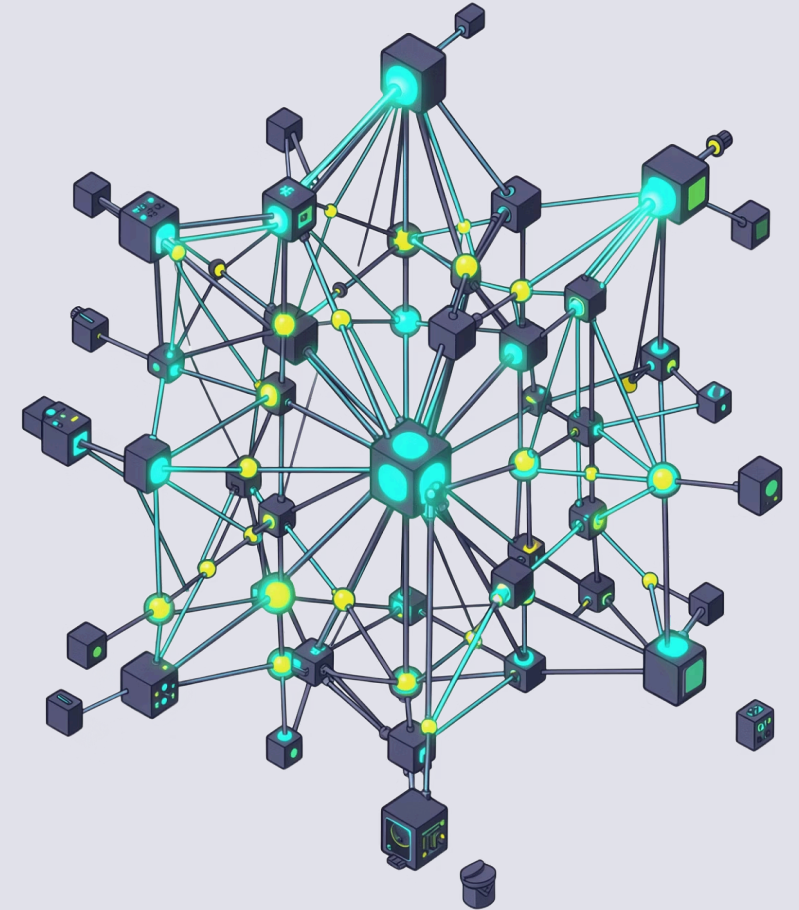


Vector Databases: A Complete Guide for Computer Engineering

A comprehensive lecture for B.Tech Computer Engineering students — covering motivation, architecture, algorithms, implementation, and real-world applications.

Faculty: Dr. Anilkumar V. Brahmane



Why Traditional Databases Fall Short

Structured Data Only

Relational DBs store structured rows & columns — great for exact lookups, but **terrible for semantic similarity**. SQL cannot express "find me something that means the same thing."

The Cat Photo Problem

Searching "find images similar to this cat photo" is **impossible with SQL** — there is no column to match against. You need an entirely new paradigm built around meaning, not equality.

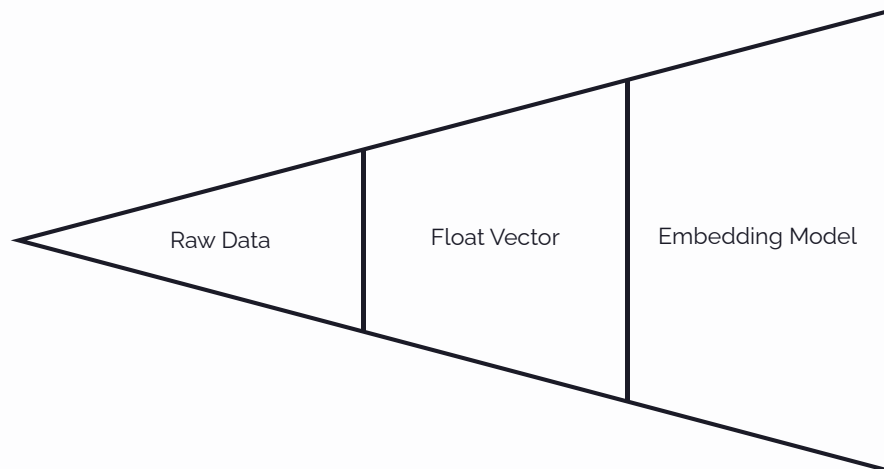
Embeddings Are Alien to SQL

AI/ML models produce **embedding vectors** — arrays of hundreds of floats that encode meaning. Traditional databases cannot index or query them efficiently at scale.

- ❏ Vector database management systems have emerged as a critical component in modern data management, driven by the need to describe rich data — texts, images, video — for recommender systems, similarity search, and chatbots.

How Vector Databases Work

The Processing Pipeline



How It Works — Simple Example

Imagine storing **1 million product descriptions**. When a user searches "*comfortable running shoes*", the system:

01

Converts the query to a **vector embedding**

02

Finds the **5 nearest vectors** in the database using ANN search

03

Returns the most **semantically similar products** — even if they don't share exact keywords!

Unlike SQL's exact match, vector DBs find **meaning-based** matches:

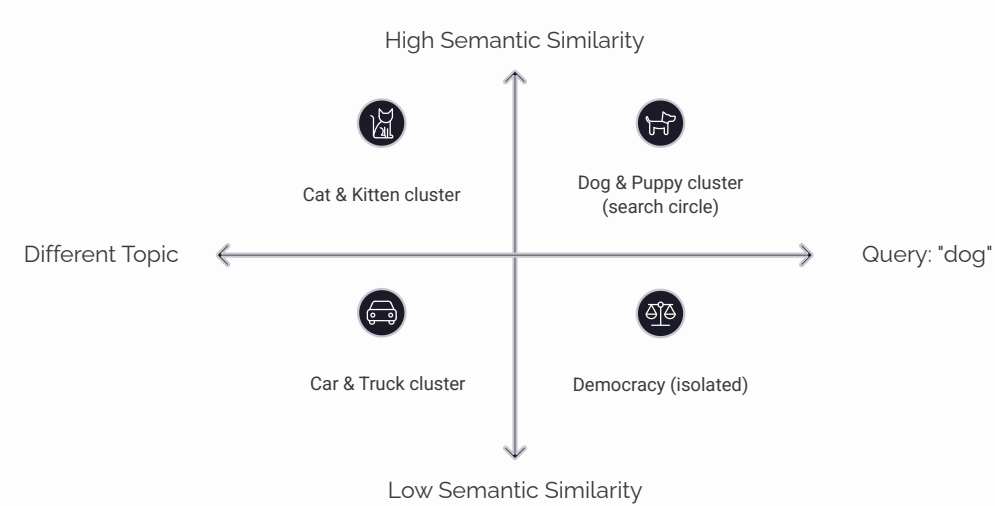
Traditional DB

Keyword match only — misses synonyms & context

Vector DB

Semantic match — understands *meaning* & intent

Similarity Search: Finding What's Close in Vector Space



Query: 'dog' → finds 'puppy', 'canine', 'hound' even without exact keyword match

What is Similarity Search?

Similarity search finds items that are CLOSEST in meaning to your query – not just exact keyword matches.

Like Spotify's 'Recommended Songs' – it finds music that FEELS similar to what you like, even if the genre or artist is different.

How it works:

01

Convert query to vector (embed it)

02

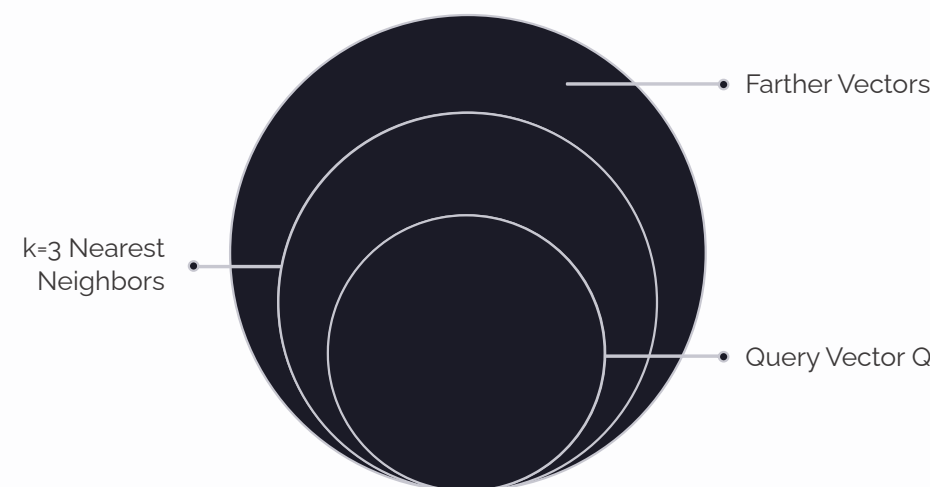
Measure distance to all stored vectors

03

Return top-k closest vectors as results

	Keyword Search	Similarity Search
Method	Exact word match	Vector distance
Example	"dog food" finds only "dog food"	"dog food" also finds "pet nutrition", "puppy diet"
Used in	SQL LIKE	Vector DB ANN

Nearest Neighbor Search (KNN vs ANN)



KNN returns the k closest points to Q

KNN (Exact)

Checks ALL points, 100% accurate, slow at scale

ANN (Approximate)

Checks a smart subset, ~95% accurate, 1000x faster

K-Nearest Neighbor Search

Given a query vector Q, find the K vectors in the database that are most similar (closest distance) to Q.

i K=3 means: return the 3 most similar items. Like asking: 'Find the 3 most similar movies to Inception.'

Why ANN instead of exact KNN?

At 1 million vectors: KNN takes ~2 seconds per query

ANN (HNSW) takes ~2 milliseconds – 1000x faster!

Accuracy loss: only ~2-5% – acceptable for most applications

Key formula box:

Given query Q, find top-k vectors $v_1, v_2, \dots, v_k$ such that $\text{distance}(Q, v_i) \leq \text{distance}(Q, v_{i+1})$

PQ (Product Quantization): Compressing Vectors to Save Memory



Memory reduced by ~16x with only ~5% accuracy loss

What is Product Quantization?

- Like ZIP compression for vectors. Instead of storing the exact float values, we store a short code that points to the closest pre-computed value in a 'codebook'.

How it works:

01

Split: Divide the high-dim vector into M equal sub-vectors

02

Cluster: Run k-means on each sub-vector space → build a codebook of centroids

03

Encode: Replace each sub-vector with the ID of its nearest centroid

Benefits vs Tradeoffs:

Benefits

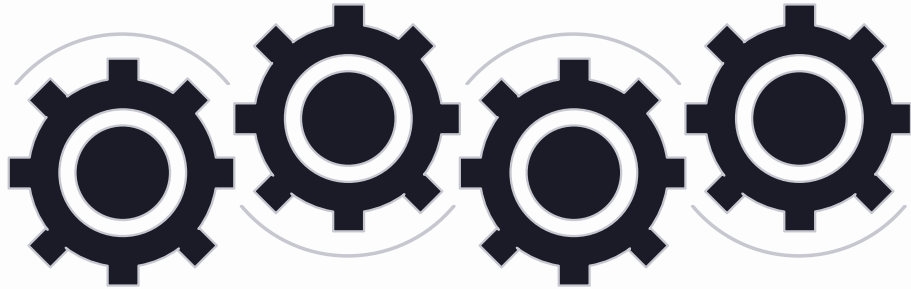
- 16-64x memory reduction
- Faster distance computation
- Enables billion-scale search

Tradeoffs

- Slight accuracy loss (~3-5%)
- Requires training phase to build codebook

Common combo: IVF-PQ = cluster with IVF + compress with PQ
→ best of both worlds

ScaNN: Google's Scalable Nearest Neighbor Library



ScaNN = Partition → Score → Rescore

What is ScaNN?

ScaNN (Scalable Nearest Neighbors) is Google's open-source ANN library, designed for extreme speed on large datasets.

i ScaNN introduces 'Anisotropic Quantization' – it compresses vectors in a smarter way by prioritizing directions that matter most for the query, not just minimizing overall error.

2X

faster than FAISS on 1M vectors

10X

faster than brute-force

—

Used in Google Search, YouTube recommendations

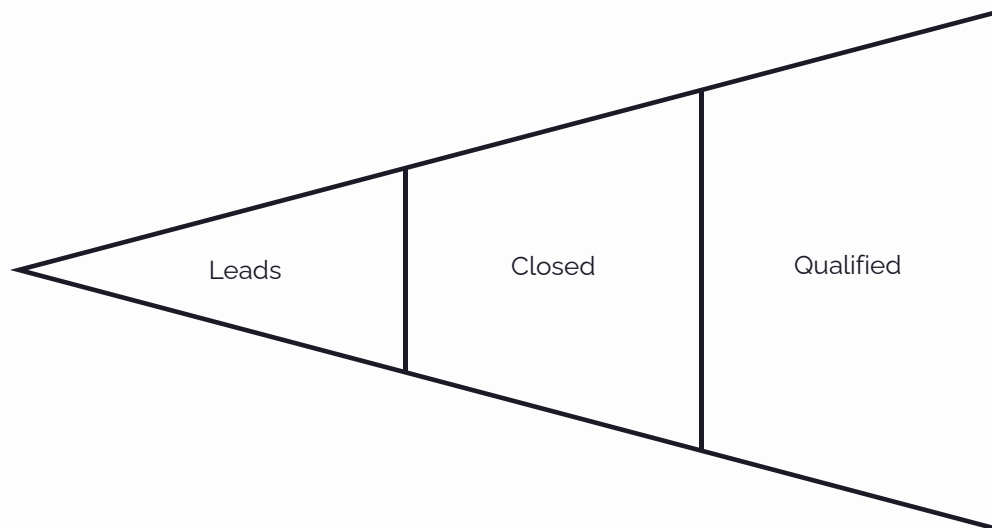
FAISS: Meta's library, great general-purpose ANN

ScaNN: Google's library, best-in-class speed on CPU

HNSW: Graph-based, best recall, higher memory

When to use ScaNN: Large-scale production systems where query latency < 5ms is required

Metadata Filtering & Hybrid Search



Hybrid Search = Vector Similarity + Metadata Filters

What is Metadata Filtering?

Metadata is extra information stored alongside each vector — like tags, categories, dates, prices. Filtering lets you combine vector search with structured conditions.



Query: "comfortable running shoes" (vector search)

+ **Filter:** brand="Nike" AND price < 3000 AND in_stock=True

→ Returns only Nike shoes under ₹3000 that are semantically similar to the query!

Pre-filtering

Apply metadata filter FIRST, then search vectors in the subset. Faster but may miss results if filter is too strict.

Post-filtering

Search vectors FIRST, then filter results. Better recall but may return fewer than k results.

Hybrid Search: Combines dense vector search (semantic) + sparse keyword search (BM25/TF-IDF) for best of both worlds. Used in enterprise search engines.

CRUD Operations in Vector Databases



There was an error generating this image

CRUD in Vector Databases

1. CREATE (Upsert)

Insert a new vector with ID and metadata

```
client.upsert("collection", points=[PointStruct(id=1,
vector=[0.1, 0.5, ...], payload={"text": "hello"})])
```

2. READ (Search/Get)

Find top-k similar vectors

```
results = client.search("collection", query_vector=
[0.1, 0.5, ...], limit=5)
```

Retrieve a specific vector by its ID

```
point = client.retrieve("collection", ids=[1, 2, 3])
```

3. UPDATE

Re-embed updated content and upsert with same ID (overwrites)

```
client.upsert("collection", points=[PointStruct(id=1,
vector=[new_vector], payload={"text":
"updated"})])
```

4. DELETE

Remove a vector by ID

```
client.delete("collection",
points_selector=PointIdsList(points=[1, 2]))
```

i Unlike SQL, there's no partial UPDATE of a vector – you must re-embed and replace the entire vector when content changes.

Chunking: Breaking Documents into Searchable Pieces



Chunking = splitting big docs into bite-sized searchable pieces

Why Chunking is Necessary

- ❗ Embedding models have a token limit (e.g., BERT = 512 tokens). A 50-page PDF can't be embedded as one vector – it must be split first.

Chunking strategies:

Fixed Size

Split every N words (e.g., 300 words). Simple but may cut sentences mid-way.

Sentence-based

Split at sentence boundaries. Preserves meaning better.

Overlapping

Each chunk overlaps with the next by ~50 words. Prevents losing context at boundaries.

Example:

Original: "Vector databases store embeddings. They support ANN search. HNSW is the most popular index."

Chunk 1: "Vector databases store embeddings. They support ANN search."

Chunk 2: "They support ANN search. HNSW is the most popular index." (50% overlap)

Key insight: Chunk size affects quality – too small loses context, too large loses precision.

Re-ranking: Improving Search Quality After Retrieval



2-Stage: Retrieve Many → Re-rank to Best

What is Re-ranking?

Re-ranking is a second pass that re-scores the top-k candidates from vector search using a more powerful (but slower) model to improve final result quality.

i ANN search is fast but approximate. The top-10 results may not be in perfect order. A cross-encoder re-ranker reads the query **AND** each result together to give a more accurate relevance score.

Bi-encoder (Vector DB)

Embeds query and docs separately → fast but less accurate

Cross-encoder (Re-ranker)

Reads query + doc together → slow but highly accurate

Common re-rankers: Cohere Rerank API, BGE-Reranker, ms-marco-MiniLM

When to use: RAG pipelines, enterprise search, legal/medical document retrieval where precision matters

Performance Metrics: How to Evaluate a Vector Database

95%

Recall@10

1K

Queries/sec (QPS)

8ms

Latency p99

120s

Index Build Time (for 1M vectors)

Key Performance Metrics

Recall@k

"Of the true k nearest neighbors, what fraction did ANN actually return?"

Formula: $\text{Recall@k} = |\text{ANN results} \cap \text{True results}| / k$

Example: ANN returns [A,B,C,D,E], True top-5 = [A,B,C,F,G]
→ $\text{Recall@5} = 3/5 = 60\%$

QPS (Queries Per Second)

How many queries the system handles per second. Higher = better throughput.

Latency (p50/p99)

p50 = median query time, p99 = worst-case for 99% of queries. Target: $\text{p99} < 10\text{ms}$ for production.

Index Build Time

Time to build the ANN index from scratch. HNSW is slow to build but fast to query.

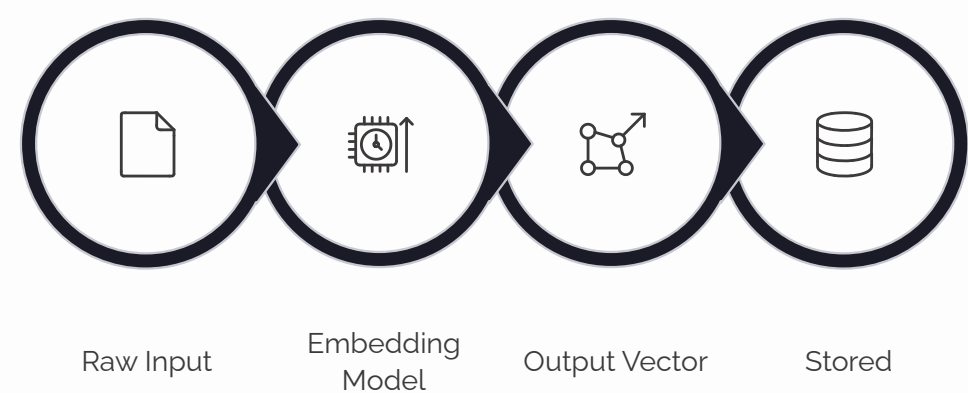
Memory Usage

RAM consumed by the index. HNSW uses 4-8x raw vector size; IVF-PQ uses much less.

i The ANN Trilemma: You can optimize for Speed, Recall, or Memory — but not all three at once. Choose based on your use case.

Embeddings: Turning Words & Images into Numbers

The Embedding Pipeline



Embeddings capture **semantic meaning** as numbers — similar inputs land **close together** in vector space, dissimilar ones land **far apart**.

"king" → [0.9, 0.2, 0.7]
Close to "queen" → [0.85, 0.25, 0.72]

"apple" → [0.1, 0.9, 0.1]
Far from "king" and "queen"

Why Embeddings Matter

GPS Analogy: Think of embeddings like GPS coordinates for *meaning*. Just like two nearby cities have similar coordinates, two similar words have similar vectors.

Fixed Size

Every input → same-length vector (e.g., 384 or 1536 numbers)

Semantic Meaning

Similar things → similar vectors, regardless of exact wording

Universal

Works for text, images, audio, video, and code

Popular Embedding Models

Model	Input	Dimensions
BERT	Text	768
OpenAI ada-002	Text	1536
CLIP	Image + Text	512
ResNet-50	Images	2048
Whisper	Audio	1280

Cosine Similarity: Measuring "Closeness" by Angle

Cosine Similarity

Formula:

$$\cos(\theta) = \frac{\vec{A} \cdot \vec{B}}{|\vec{A}| \times |\vec{B}|}$$

Range: -1 (opposite) → 0 (unrelated) → +1 (identical)

Real-World Example

Pair	Vectors (simplified)	Score
"king" vs "queen"	[0.9, 0.1] vs [0.85, 0.15]	≈ 0.99 ✔ Very Similar!
"king" vs "apple"	[0.9, 0.1] vs [0.1, 0.9]	≈ 0.18 ✗ Very Different!



Small angle θ → **similar** vectors | Large angle θ → **different** vectors

ⓘ Cosine similarity measures the **angle** between vectors – not their length. So a long document and a short document on the same topic still score high!

Metric Comparison

Cosine Similarity
Use for text embeddings – angle-based, length-independent
Euclidean Distance
Use for images / audio – straight-line distance in space
Dot Product
Use for normalized vectors – fastest computation

Euclidean Distance & Dot Product — Explained Simply

Euclidean Distance (L2)

$$d(A, B) = \sqrt{\sum (A_i - B_i)^2}$$

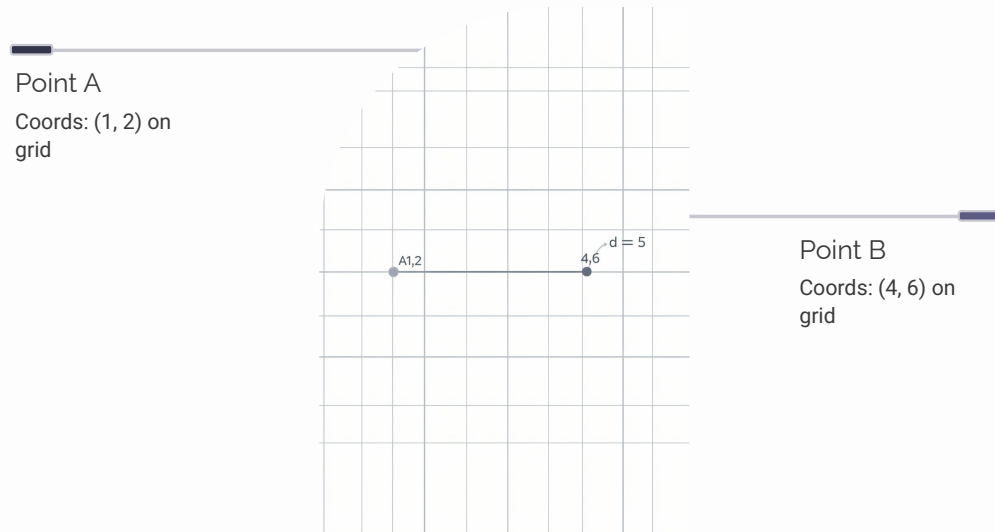
It's the straight-line ruler distance between two points on a map.

Example:

Point A = [1, 2], Point B = [4, 6]

$$d = \sqrt{((4-1)^2 + (6-2)^2)} = \sqrt{(9+16)} = \sqrt{25} = 5$$

Use case: Image embeddings, audio features



Dot Product

$$A \cdot B = \sum (A_i \times B_i)$$

Measures how much two vectors point in the same direction AND how large they are.

Example:

A = [1, 2, 3], B = [4, 5, 6]

$$A \cdot B = (1 \times 4) + (2 \times 5) + (3 \times 6) = 4 + 10 + 18 = 32$$

Use case: Normalized vectors, fast retrieval

Key difference from cosine: Dot product is affected by vector magnitude; cosine is not.

Cosine Similarity

Best for: Text, NLP

Euclidean Distance

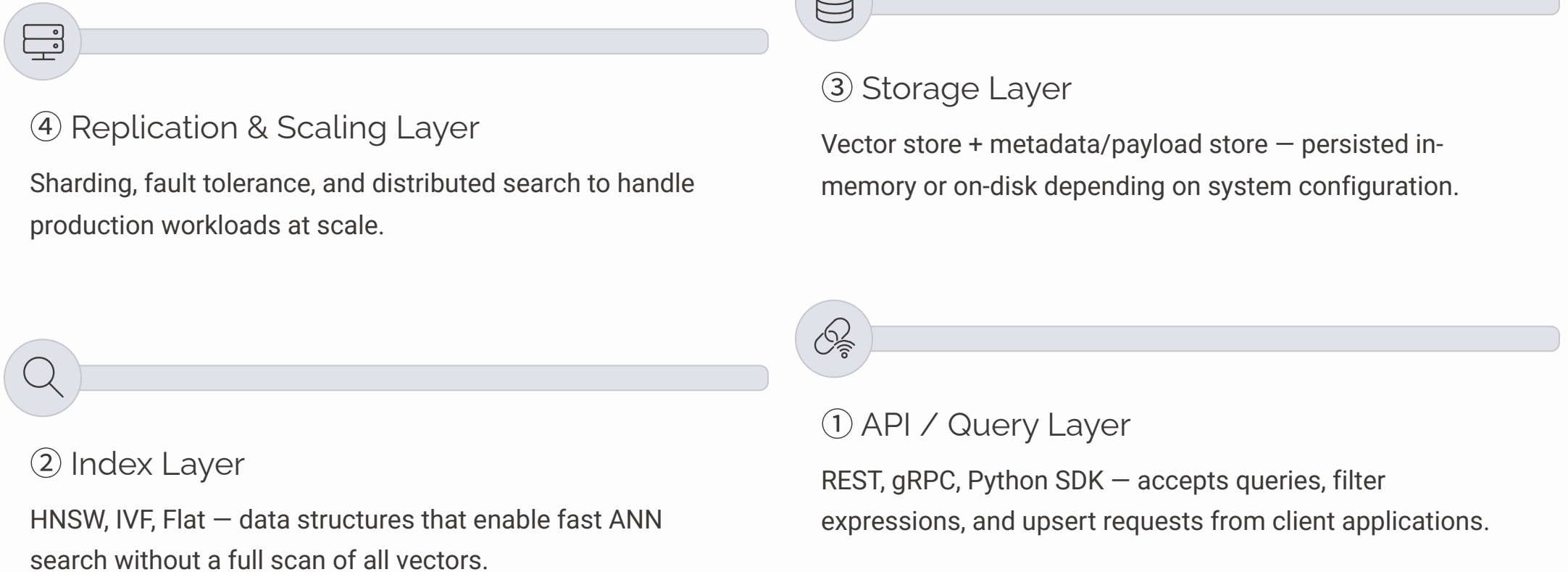
Best for: Images, Audio

Dot Product

Best for: Normalized vectors, Speed

Architecture Overview: The 4 Layers of a Vector Database

Data flows ↓ **down** through the layers on **write**, and ↑ **up** through the layers on **query** – each layer adding a distinct capability.



Ingestion Pipeline



① Embed

Raw data (text / image) → Embedding Model → Float vector of fixed dimensionality



② Write

Vector + metadata written to **Write-Ahead Log (WAL)** for durability before any index update



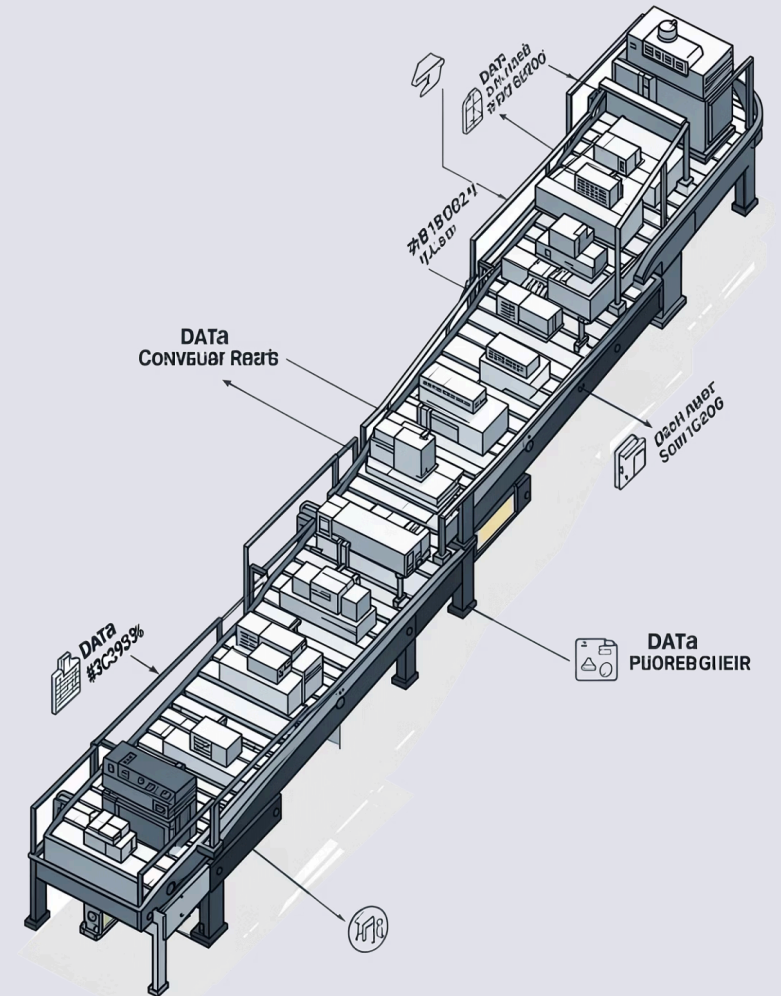
③ Index

New vector inserted into ANN index — e.g., new node connected into the **HNSW graph**



④ Persist

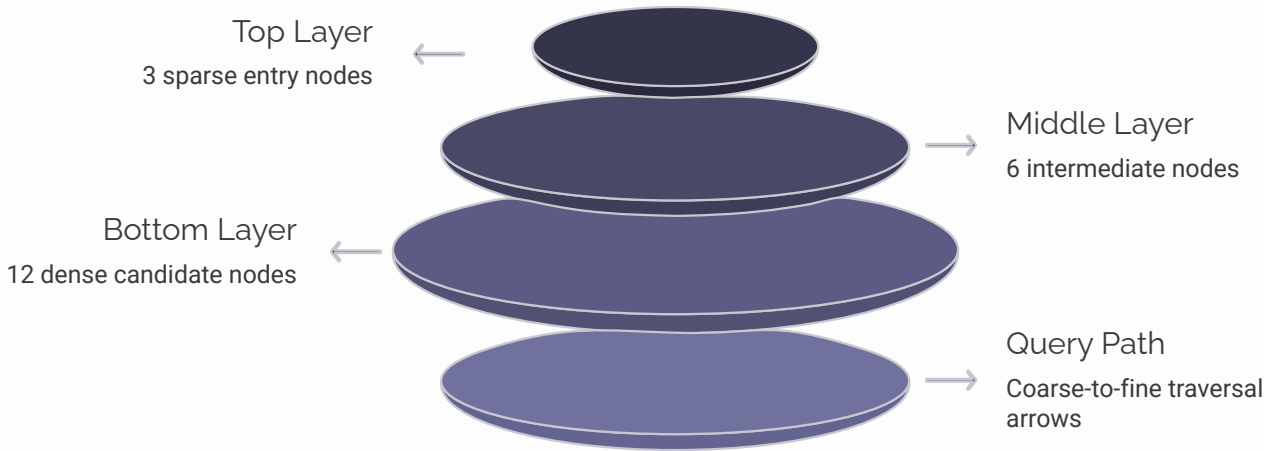
Vectors flushed to disk; periodic **compaction** optimizes index structure for query performance



Indexing Algorithms: HNSW vs IVF

Algorithm Comparison

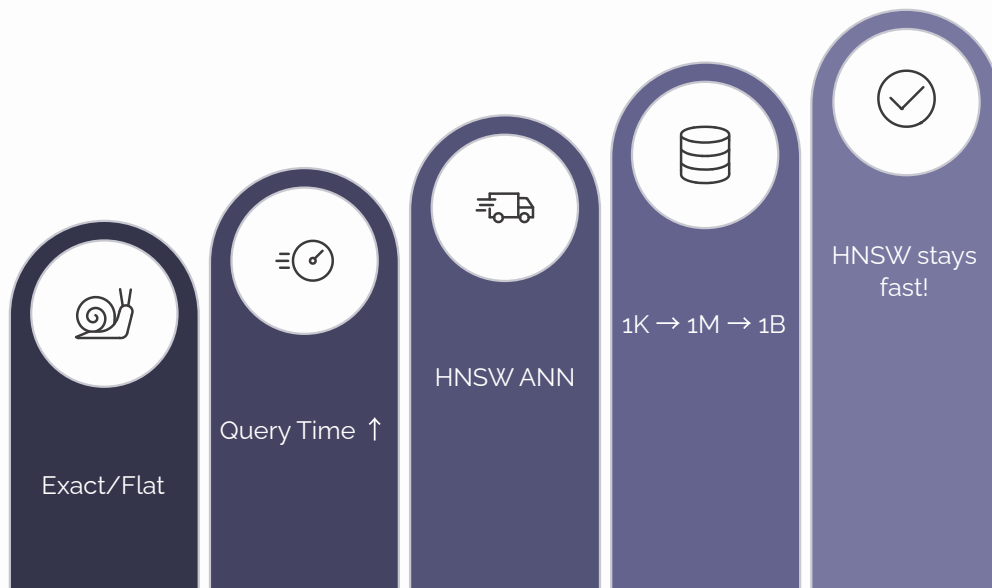
Algori thm	Appro ach	Recal l	Mem ory
HNS W	Graph layers (coar se→fi ne)	★★★ ★ High	High (4– 8×)
IVF	k- mean s cluste r probi ng	★★★ Medi um	Mode rate
Flat	Brute- force exact scan	★★★ ★ 100%	Low (no index)
IVF- PQ	Clust ers + comp ressio n	★★★ Medi um	Very Low



HNSW navigates from a **coarse top layer** to a **fine-grained bottom layer**, skipping irrelevant regions of vector space for fast approximate search.

❏ Flat search is only feasible for <100K vectors. At billion scale, HNSW or IVF-PQ are the industry standard choices.

ANN Search: Why Exact Search Doesn't Scale



The Scalability Problem

Exact search (Brute Force): checks ALL vectors → $O(n \cdot d)$ time
→ 1 billion vectors × 1536 dims = too slow!

ANN (Approximate Nearest Neighbor): trades tiny accuracy loss for massive speed gain

HNSW example: 1 billion vectors, query time < 10ms, recall > 95%

01

Enter at top layer (few nodes, fast navigation)

02

Greedy move to nearest neighbor

03

Drop to next layer, repeat

04

At bottom layer, return top-k candidates

ANN is the core innovation that makes vector databases practical at scale.

Query Lifecycle: From API Call to Result



Embed the Query

Client sends query text → embedding model converts it to a **query vector** of the same dimensionality as the stored vectors.



Send to API Layer

Query vector sent to the API layer along with optional **metadata filters** (e.g., `category = "science", date > 2023`).



ANN Index Traversal

Index layer traverses **HNSW graph** or probes **IVF clusters** to efficiently find the top-k candidate vectors.



Apply Metadata Filter

Metadata filters applied — either **pre-filtering** (reduces candidate pool first) or **post-filtering** (applied after ANN retrieval).



Return Ranked Results

Vector IDs, metadata payloads, and **similarity scores** returned to client, ranked by closeness to query vector.

Types of Vector Databases



Purpose-Built Standalone

Pinecone, Weaviate, Qdrant, Milvus — designed from scratch for vector search. Best performance and richest feature sets for AI-native applications.



Extended Relational

pgvector (PostgreSQL extension) — adds ANN vector search to existing SQL databases. Ideal if you already use PostgreSQL in production.



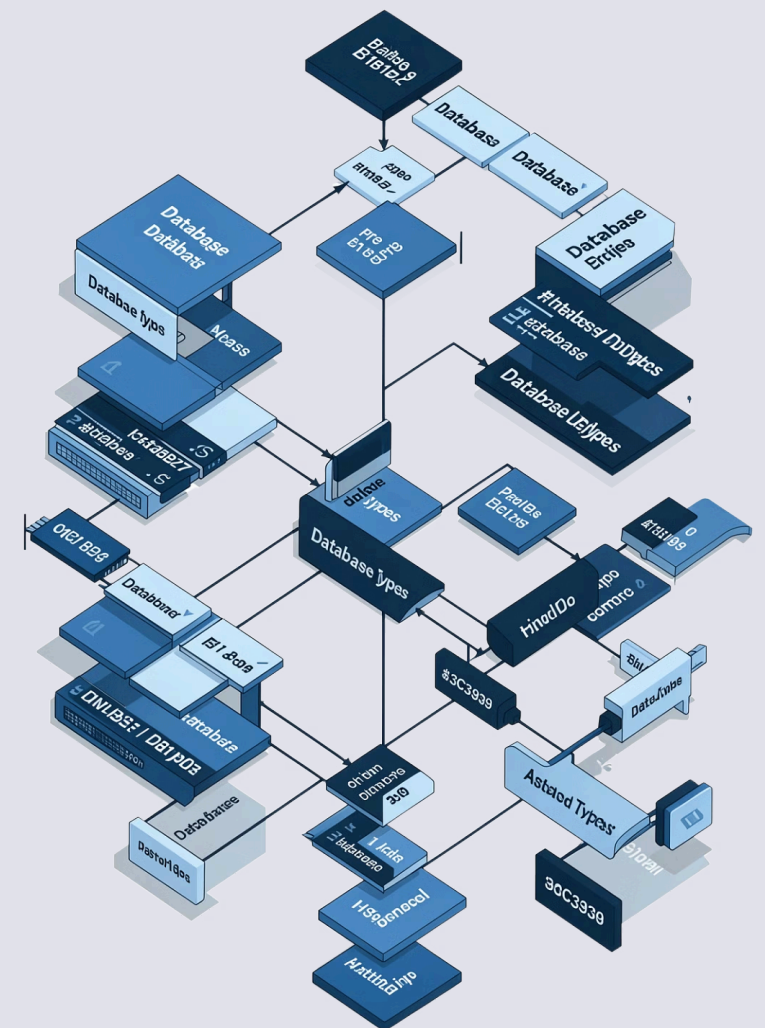
Extended NoSQL

MongoDB Atlas Vector Search, Redis Vector — vector capability bolted onto document or key-value stores. Convenient for teams already using these platforms.



Vector Libraries

FAISS, Annoy, ScaNN — in-memory ANN libraries with no persistence or CRUD. Best for research, prototyping, and embedding into custom systems.



Popular Vector Databases Compared

Database	Type	Index Support	Best For
Pinecone	Managed Cloud	HNSW	Production RAG apps
Weaviate	Open Source	HNSW	Multi-modal search
Qdrant	Open Source	HNSW + IVF	High-performance search
Milvus	Open Source	HNSW, IVF, PQ	Billion-scale vectors
pgvector	SQL Extension	IVF, HNSW	Existing PostgreSQL users
FAISS	Library	IVF, PQ, Flat	Research & prototyping

Choosing between these systems depends on your scale requirements, infrastructure preferences, and whether you need managed hosting or self-hosted control.

FAISS: Facebook AI Similarity Search — Deep Dive

What is FAISS?

FAISS (Facebook AI Similarity Search) is a free, open-source C++ library (with Python bindings) for fast similarity search and clustering of dense vectors. It's NOT a full database — it's a library you embed in your code.

Speed

Searches 1 billion vectors in milliseconds using GPU

Multiple Indexes

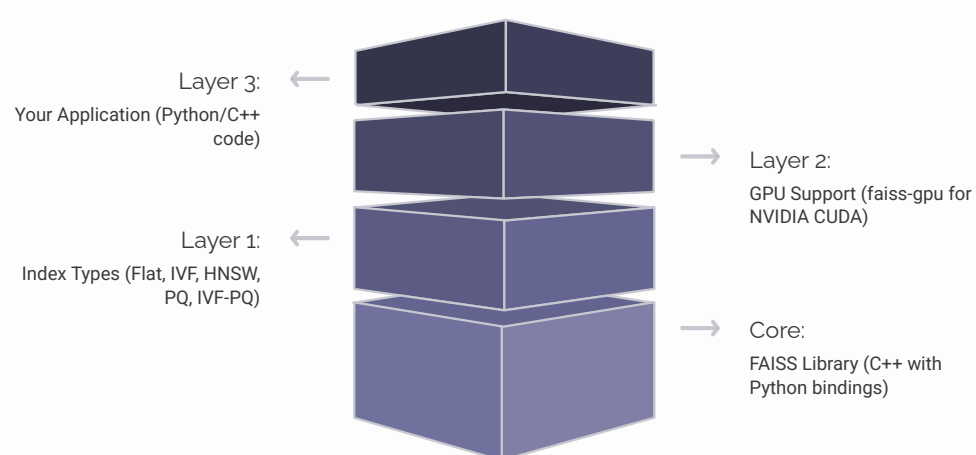
Flat, IVF, HNSW, PQ, IVF-PQ — choose based on your needs

GPU Support

faiss-gpu uses NVIDIA CUDA for 10-100x speedup

Free & Open Source

Used by Meta, Spotify, Airbnb in production



FAISS = In-memory ANN library by Meta (Facebook AI Research)

Simple Python example:

```
import faiss, numpy as np
d = 128 # vector dimension
index = faiss.IndexFlatL2(d) # exact L2 search
vectors = np.random.rand(10000, d).astype('float32')
index.add(vectors) # add 10,000 vectors
query = np.random.rand(1, d).astype('float32')
D, I = index.search(query, k=5) # find top-5
print("Nearest IDs:", I) # [3421, 7823, 1204, 9012, 456]
```

⚠ FAISS has NO persistence (data lost on restart), NO metadata filtering, NO REST API, NO CRUD. For production apps, use Qdrant/Weaviate which use FAISS internally.

Pinecone: Managed Vector Database — Deep Dive

What is Pinecone?

Pinecone is a fully managed, cloud-native vector database. You don't install anything — just create an account, get an API key, and start storing vectors. Pinecone handles all infrastructure, scaling, and maintenance.

Serverless

No servers to manage. Auto-scales from 0 to billions of vectors.

Fast

Sub-10ms queries at any scale using proprietary HNSW-based index

Namespaces

Logical partitions within one index (e.g., separate user data)

Hybrid Search

Supports both dense (semantic) + sparse (keyword) search

Simple Python example:

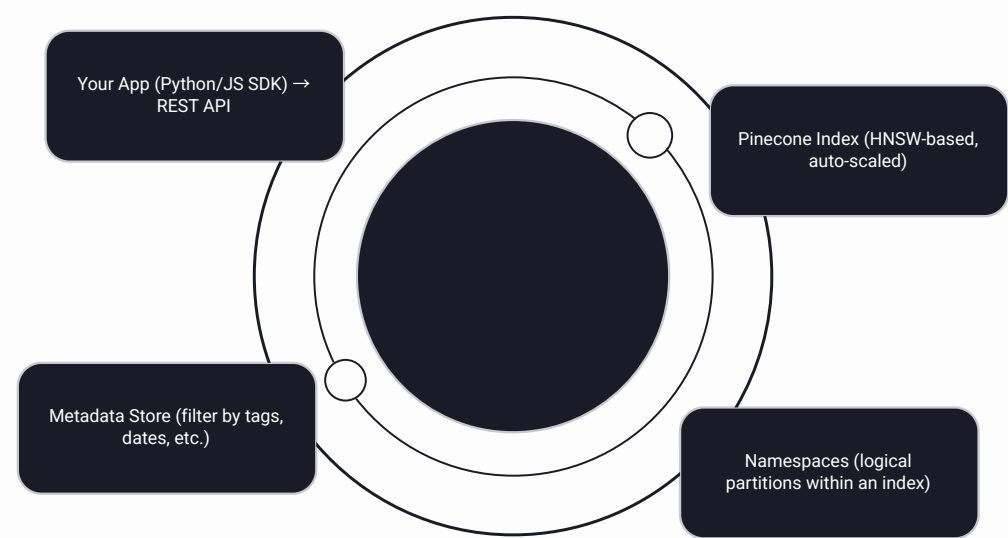
```
from pinecone import Pinecone
pc = Pinecone(api_key="YOUR_API_KEY")
index = pc.Index("my-index")
# Upsert vectors
index.upsert(vectors=[
    ("id1", [0.1, 0.5, 0.3, ...], {"category": "tech"}),
    ("id2", [0.9, 0.2, 0.7, ...], {"category": "sports"})
])
# Query
results = index.query(vector=[0.1, 0.5, 0.3, ...], top_k=5,
    filter={"category": "tech"})
```

FAISS

Library, self-managed, free, no persistence, research use

Pinecone

Managed cloud, paid, persistent, production-ready, enterprise use



Pinecone = Zero-ops, fully managed vector DB in the cloud

Implementation: Storing & Querying with Python (Qdrant)

Step-by-Step Workflow

1 Install Dependencies

```
pip install qdrant-client sentence-transformers
```

2 Embed Text

Use `SentenceTransformer('all-MiniLM-L6-v2')` to convert text → 384-dimensional float vector

3 Upsert Vectors

Insert vectors with IDs and metadata payload into a named Qdrant collection

4 Search

Pass query vector → get top-k results with similarity scores and metadata payloads returned

```
from qdrant_client import QdrantClient
from qdrant_client.models import PointStruct
from sentence_transformers import
SentenceTransformer

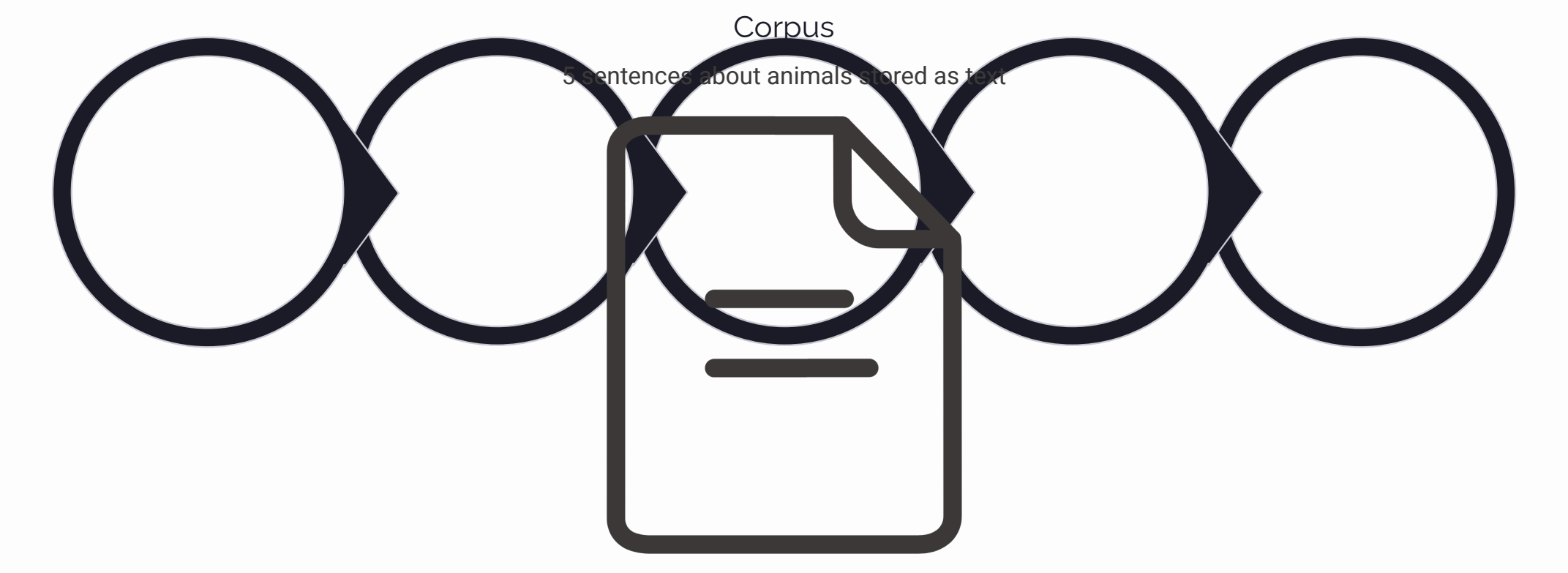
# Step 1: Connect to local Qdrant instance
client = QdrantClient(host="localhost", port=6333)
model = SentenceTransformer('all-MiniLM-L6-v2')

# Step 2: Embed documents
docs = ["Python is great", "Vector DBs are fast"]
vectors = model.encode(docs).tolist()

# Step 3: Upsert into collection
points = [
    PointStruct(id=i, vector=v, payload={"text": d})
    for i, (v, d) in enumerate(zip(vectors, docs))
]
client.upsert(collection_name="demo", points=points)

# Step 4: Query
query_vec = model.encode(["fast search"]).tolist()[0]
results = client.search(
    collection_name="demo",
    query_vector=query_vec,
    limit=2
)
for r in results:
    print(r.payload["text"], r.score)
```

End-to-End Example: Semantic Search Engine



```
from sentence_transformers import
SentenceTransformer
from qdrant_client import import QdrantClient
from qdrant_client.models import import VectorParams,
Distance, PointStruct

# 1. Setup
model = SentenceTransformer('all-MiniLM-L6-v2')
client = QdrantClient(":memory:")

# 2. Create collection
client.create_collection("animals",
    vectors_config=VectorParams(size=384,
distance=Distance.COSINE))

# 3. Embed & store
sentences = [
    "The cheetah is the fastest land animal",
    "Dolphins are highly intelligent marine mammals",
    "Eagles soar high above mountain peaks",
    "Tigers hunt prey in dense jungle forests",
    "Penguins waddle across Antarctic ice"
]
vectors = model.encode(sentences)
client.upsert("animals", points=[
    PointStruct(id=i, vector=v.tolist(), payload={"text": s})
    for i, (v, s) in enumerate(zip(vectors, sentences))
])

# 4. Query
query_vec = model.encode(["fast jungle predator"])
results = client.search("animals",
query_vector=query_vec[0], limit=2)
for r in results:
    print(r.payload["text"], "→ score:", round(r.score, 3))
```

Query: "fast jungle predator"

Result 1: "The cheetah is the fastest land animal" → score: 0.847

Result 2: "Tigers hunt prey in dense jungle forests" → score: 0.791

No keyword 'cheetah' or 'tiger' in the query — yet the DB found them by MEANING!

Real-World Applications



RAG — Retrieval-Augmented Generation

LLMs like ChatGPT retrieve relevant document chunks from a vector DB before generating answers — dramatically **reducing hallucinations** and keeping responses grounded in real data.



Semantic Search

Google-style search that understands **meaning, not just keywords**. Finds documents about "automobile accidents" when you search "car crash" — no keyword overlap needed.



Recommendation Systems

Netflix, Spotify — "**find items similar to what this user liked.**" User preference vectors are matched against item vectors in real time at millions of requests per second.



Anomaly Detection

Cybersecurity and fraud detection — vectors that are **far from normal clusters** are flagged for review. Enables behavioral-pattern-based threat detection without explicit rules.



RAG Architecture: Vector DB + LLM



Semantic Retrieval

Query is embedded and matched against document chunk vectors. Top-k most relevant passages retrieved in milliseconds.

Context Injection

Retrieved chunks are injected into the LLM prompt as grounding context — enabling accurate, citation-backed answers.

No Retraining Needed

New knowledge is added by **updating the vector DB** only — the LLM itself never needs to be retrained on new data.

[illegible]

Faculty: Dr. Anilkumar V. Brahmane

Key Takeaways for B.Tech Students



The Memory Layer of Modern AI

Vector DBs are not optional infrastructure — **every production LLM application uses one**. Mastering this is mastering the foundation of applied AI engineering.

The Core Skill Chain

Understand the full pipeline: **Embeddings** → **Similarity Metrics** → **ANN Indexing** → **Query Lifecycle**. Each link is a fundamental CS concept worth studying deeply.

Start Hands-On Today

Run **Qdrant locally**, embed 1,000 Wikipedia sentences with SentenceTransformers, and build your own semantic search engine — a project that belongs on your résumé.

Research Frontier

Open problems await: **billion-scale search, multi-modal vectors, GPU-accelerated indexing**. This field is evolving fast — the papers being written today will define systems of 2030.

Sources & References

- Vector database management systems: Fundamental concepts, use-cases, and current challenges
- Vector Database Management Techniques and Systems – SIGMOD 2024 Tutorial
- Vector Database Management – Tsinghua University, 2024
- How Vector Databases Work Internally – Krunal Kanojiya
- High-level architecture of a Vector Database – About Vector Database

Faculty: Dr. Anilkumar V. Brahmane